Matière : Dev. App. Rép.
Niveau d'études : L3 Info

Enseignante : Malek Ltaief
Année universitaire : 2026-2027

TP2 : Programmation réseaux avec les sockets UDP

Objectif du TP

Ce TP a pour objectif de manipuler la communication client/serveur via les sockets UDP en Java.

Rappel : Les Sockets UDP

Le protocole UDP utilise un mode de communication non connecté. Chaque datagramme envoyé est indépendant, sans garantie :

— ni d'arrivée,

— ni d'ordre de réception.

La seule garantie est que les données reçues ne sont pas altérées.

Schéma général

Émission :

1. Création d'un socket UDP : `new DatagramSocket()`
2. Création du paquet : `new DatagramPacket()`
3. Envoi via : `socket.send()`

Réception :

1. Création du socket lié au port d'écoute
2. Préparation d'un paquet vide
3. Réception blocante : `socket.receive()`
4. Extraction des données

Exercice

Copier puis tester sur la même machine, puis sur deux machines différentes, les programmes suivants.

Code ClientUDP.java

```
import java.net.*;
import java.util.Scanner;
public class ClientUDP {
public static void main(String argv[]) {
int port =5123,port1=2341;
String host = "";
String msg1;
Scanner keyb = new Scanner(System.in);
try {
// on récupère les paramètres : nom de la machine serveur
System.out.println("Adress du serveur : ");
host = keyb.next();
InetAddress adr;
DatagramPacket packet;
DatagramSocket socket;
// adr contient l'@IP de la partie serveur
adr = InetAddress.getByName(host);
// données àenvoyer : chaîne de caractères
byte[] data = (new String("Hello Word")).getBytes();
// création du paquet avec les données et en précisant l'adresse du serveur (@IP et
    port sur lequel il écoute)
packet = new DatagramPacket(data, data.length, adr, port1);
// création d'une socket, sans la lier àun port particulier
socket = new DatagramSocket();
// envoi du paquet via la socket
socket.send(packet);
DatagramSocket socketRecu = new DatagramSocket(port);
// création d'un tableau vide pour la réception
byte[] reponse = new byte[15];
DatagramPacket paquetRecu= new DatagramPacket(reponse,reponse.length);
//attente paquet envoyé sur la socket du client
socketRecu.receive(paquetRecu);
//récupération et affichage de la donnée contenue dans le paquet
msg1 = new String(paquetRecu.getData(),0,paquetRecu.getLength());
System.out.println(" reçu du serveur : "+msg1);
} catch (Exception e) {
System.err.println("Erreur : " + e);
}
}
}
```

Code ServeurUDP.java

```
import java.net.*;
import java.util.Scanner;
public class ServeurUDP {
public static void main(String argv[]) {
int port =2341, port1 =5123;
String msg1;
Scanner keyb = new Scanner(System.in);
try {
// on récupère le paramètre : port d'écoute
System.out.println("server en attent");
DatagramPacket packetRecu;
// création d'une socket liée au port précisé en paramètre
DatagramSocket socket = new DatagramSocket(port);
// tableau de 15 octets qui contiendra les données reçues
byte[] data = new byte[1024];
// création d'un paquet en utilisant le tableau d'octets
packetRecu = new DatagramPacket(data, data.length);

// attente de la réception d'un paquet. Le paquet reçu est placé
// dans packet et ses données dans data.
socket.receive(packetRecu);
// récupération et affichage des données (une chaîne de caractères)
msg1 = new String(packetRecu.getData(), 0, packetRecu.getLength());
System.out.println(" le message recu est: " + msg1);
System.out.println(" ça vient de l' @: " + packetRecu.getAddress() + "et du port: " +
    packetRecu.getPort());
// on met une nouvelle donnée dans le paquet
// (qui contient donc le couple @IP/port de la socket coté client)
InetAddress ad = null;
DatagramSocket socketRecu= new DatagramSocket();
byte[] reponse = (new String("bien recu")).getBytes();
ad = InetAddress.getLocalHost();
DatagramPacket dgp = new DatagramPacket(reponse,reponse.length,ad,port1);
//on envoie le paquet au client
socketRecu.send(dgp);
} catch (Exception e) {
System.err.println("Erreur : " + e);
}
}
}
```